

hw10

Please make sure you're following all the instructions described in the "Homework Instructions" page on Canvas (go to the homepage of our class and you'll see a link saying "Homework Instructions" just underneath the header of the page).

IMPORTANT NOTE

It turns out that in R version 4.6.0 the `summary()` function was updated so that the name of the element that lists the number of NA's is now called `NAs` (it used to be called `NA's` with an apostrophe). For Question 2, you're required to use the `summary()` function. Unfortunately, this change means that if you're running an earlier version of R than 4.6.0, that slight change will make the autograder count the question as incorrect, since the output values don't match exactly (you can check R version by running `R.Version()$version.string`). If you have R 4.6.0 or later, you should be good to go. If your version of R is earlier than that, put the following code at the top of your script:

```
summary <- function(...) {  
  x <- base::summary(...)  
  names(x)[names(x) == "NA's"] <- "NAs"  
  x  
}
```

This will create an alternate version of `summary()` that will change the name so that it matches the key. I've tested this out and it appears to fix the problem, so hopefully this takes care of it.

1 Fibonacci sequence

The Fibonacci sequence is a sequence of integers where each value is sum of the two previous values, where the first two values are $F_1 = 1$ and $F_2 = 1$. The first 10 numbers of the Fibonacci sequence are:

1 1 2 3 5 8 13 21 34 55

Use a for loop to calculate the first 50 numbers of the Fibonacci sequence.

var: `fib` - numeric vector containing the first 50 numbers of the Fibonacci sequence

2 Summarizing objects

The `summary()` function can be run on any R object and it'll print out a short summary/description of the object. Consider the output for both a numeric vector and a character vector:

```
summary(1:10)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.00	3.25	5.50	5.50	7.75	10.00

```
summary(c("A", "B", "A", "B", "B", "C"))
```

Length	N.unique	N.blank	Min.nchar	Max.nchar
6	3	0	1	1

2.1 *An alternate summary function*

Suppose that I'd like more descriptive output for a character vector - instead of just giving me the length, I'd like to instead see how many times each value occurs (remember the `table()` function?). Let's write a function called `summary2` that does this for us. If the input is a character vector, it should create a table using `table()` and print out the result. If it's any other kind of object, it should use the normal `summary()` function.

var: `summary2` - function that accepts a single object and returns a `table()` if it's a character vector; otherwise, it should return the result of using `summary()` on the object.

2.2 *Modifying the function*

One downside of that approach is that character vectors with many unique values will have a large amount of output (to demonstrate this, you could try running `summary2(as.character(1:200))`). Let's modify the function so that if the input vector is a character vector and there are more than 10 unique values, the function will only return the first 10 values of the table.

var: `summary3` - same as `summary2` but with the changes described above.

2.3 *Using it on the columns of a data frame*

Use a `for` loop to run `summary3()` on each column of the built-in dataset `penguins_raw` (note that this is similar to the built-in data set `penguins` but has additional columns) and stores the result in a list. The end result should be a list where each element represents the result of summarizing one column. After you've created the list, give the elements the same names as the column names of `penguins_raw`.

var: `smry1` - list containing the column summaries as described above.

2.4 *Using lapply*

Do the same thing as above (run `summary3()` on every column) but this time do it with `lapply()`.

var: `smry2` - same as `smry1` but created using `lapply` instead of a `for` loop

3 Counting even numbers

Let's write some code to count how many even numbers are in a vector.

3.1 *Use vectorization*

This can (and generally should) be done by making use of R's vectorization, so first write a function that counts the number of even numbers in a vector *without* using a loop or an `*apply()` function.

var: `n_even1` - function that accepts a numeric vector and returns an integer that represents the number of even numbers in the vector.

3.2 *Using a for loop*

Even though it's best to do it using vectorized functions, let's do it with a for loop anyway. Write a function called `n_even2` that does the exact same thing as `n_even1` but using a for loop.

var: `n_even2` - function that does the same thing as `n_even1` but using a for loop.

4 Calculating column means

The built-in dataset `mtcars` contains data on 32 cars. Note that every column is numeric.

4.1 *for loop*

Use a for loop to calculate the mean of each column and store the means in a vector. The names of the resulting vector should be the same as the column names.

var: `mns1`

4.2 **apply*

Now do the same thing using an appropriate `*apply` function (`apply`, `sapply`, `lapply`, or `tapply`).

var: `mns2`